

Spieldetails & Design Patterns

Dieses Projekt ist eine Implementierung des Spiels "Red Light, Green Light" unter Verwendung von Pygame, entworfen für BCI (Brain-Computer Interface) Anwendungen im EEG-Kontext.

Installation & Ausführung

1. **Voraussetzungen installieren:** `bash pip install -r requirements.txt`
2. **Spiel starten:** `bash python Main.py`

Projektstruktur & Design Patterns

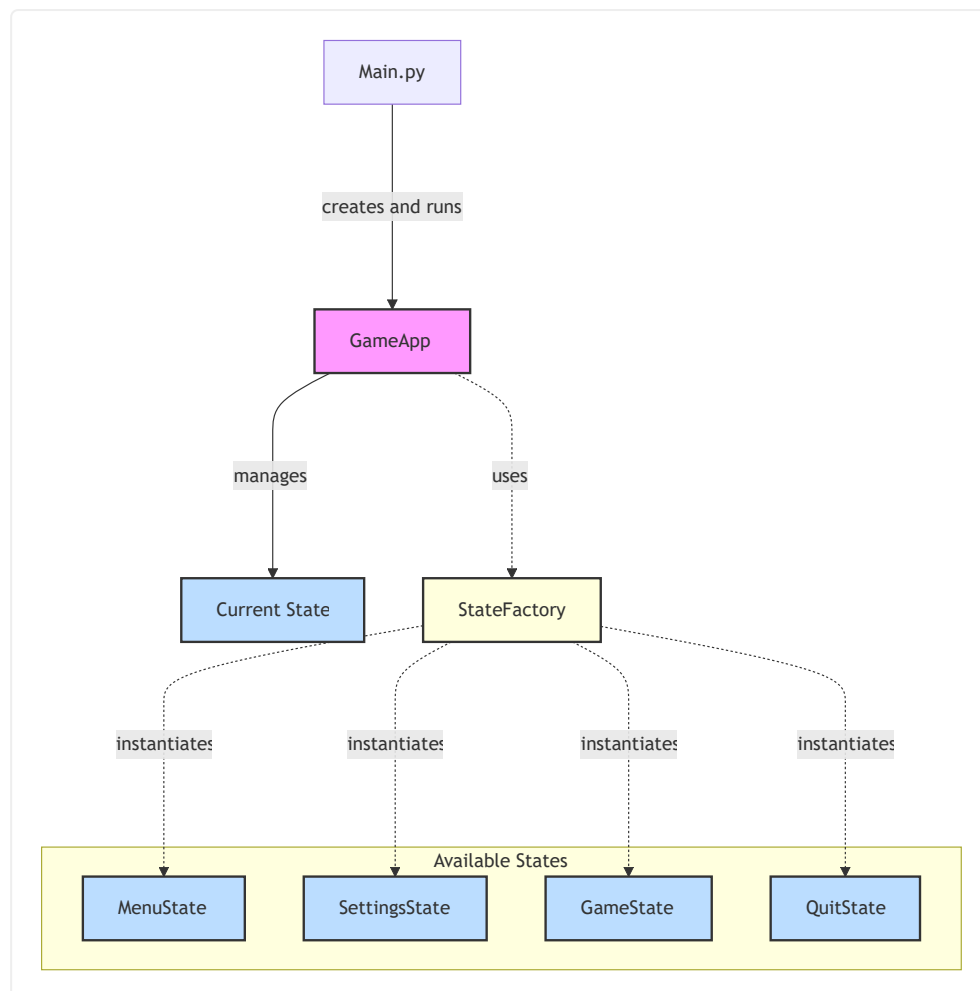
Das Projekt folgt einer klaren Struktur unter Verwendung bewährter Software-Design-Patterns, um Wartbarkeit und Erweiterbarkeit zu gewährleisten.

Verwendete Patterns

- **State Pattern:** Das Herzstück der Anwendung. `GameState`, `MenuState`, `SettingsState` und `QuitState` erben von einer abstrakten Basisklasse `State`. Jeder Zustand verwaltet sein eigenes Verhalten und Übergänge.
- **Factory Pattern:** `StateFactory` wird verwendet, um Instanzen der verschiedenen States zu erzeugen und wiederzuverwenden (Lazy Initialization).
- **Model-View-Controller (MVC):** Die States (z.B. Menu, Game) sind intern in Model (Daten), View (Anzeige) und Controller (Logik) unterteilt.
- **Observer Pattern:** Das `SettingsModel` fungiert als Subject. States registrieren sich als Observer, um bei Einstellungsänderungen (z.B. Auflösung, Sound) sofort reagieren zu können.
- **Singleton (Konzept):** `MusicManager` und Manager-Klassen werden einmalig initialisiert und durchgereicht.

Architektur-Diagramm (Übersicht)

Das folgende Diagramm zeigt die Initialisierung und die Verwaltung der Spielzustände durch die zentrale `GameApp`.

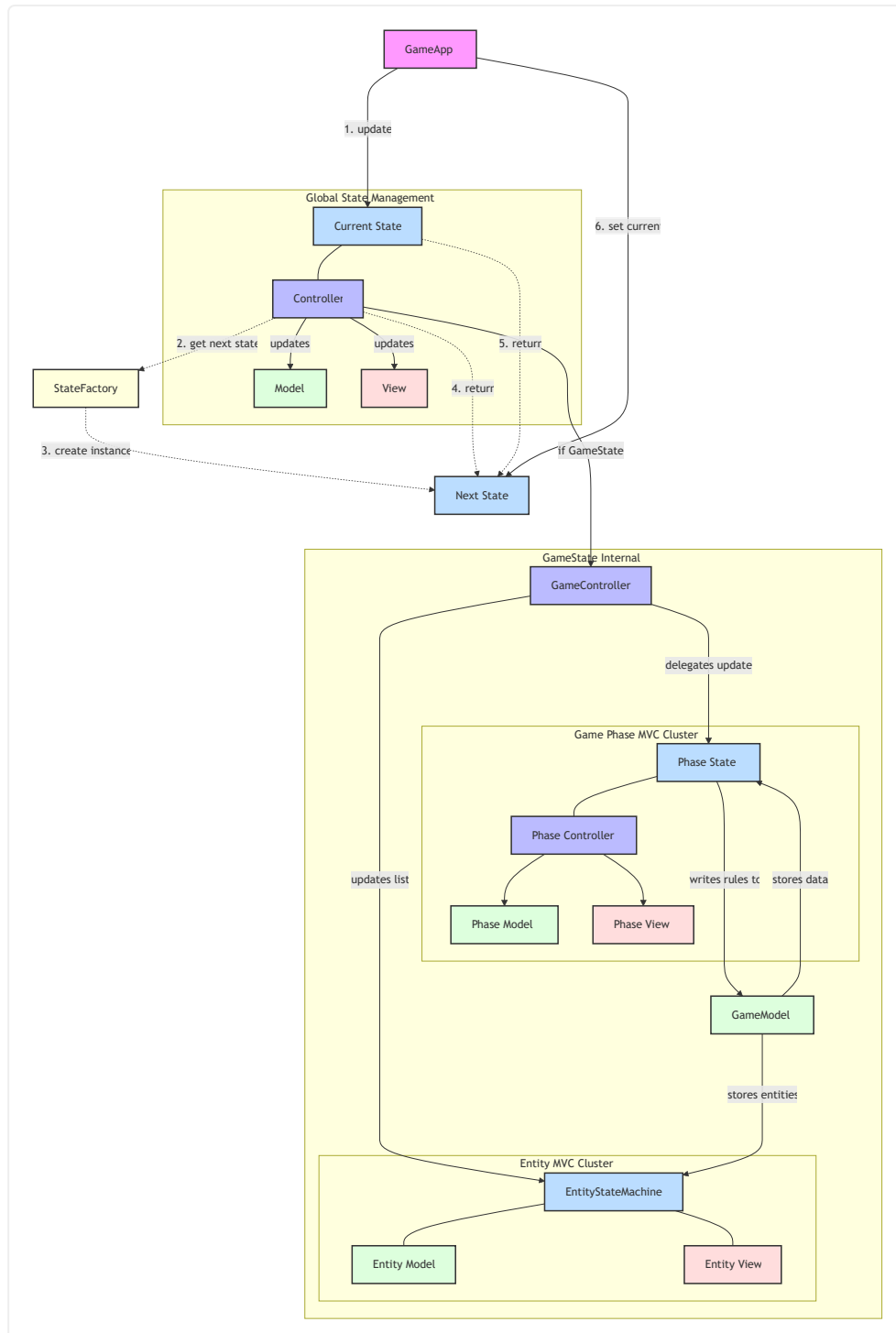


Zustandsübergangs-Lifecycle

Der Lebenszyklus der Anwendung wird durch die `GameApp` gesteuert. Ein wesentlicher Aspekt ist die delegierte Entscheidungsfindung: 1. **Update**: Die `GameApp` ruft in jedem Frame die `update()` Methode des aktuellen `State` auf. 2. **Delegation**: Der `State` delegiert die Logik an seinen internen `Controller`. 3. **Transition**: Wenn ein Zustandswechsel nötig ist (z.B. Klick auf "Start"), entscheidet der `Controller` über den nächsten Zustand und lässt diesen über die `StateFactory` erzeugen. 4. **Rückgabe**: Der Controller gibt den neuen Zustand an den `State` zurück, welcher diesen wiederum an die `GameApp` liefert. Die `GameApp` setzt diesen dann als aktuellen Zustand für den nächsten Frame.

Architektur-Details (Hierarchisches MVC)

Das Projekt nutzt eine verschachtelte **Model-View-Controller (MVC)** Struktur. Nicht nur die Haupt-Zustände, sondern auch untergeordnete Systeme wie die Spielphasen und Entities folgen konsequent diesem Muster.



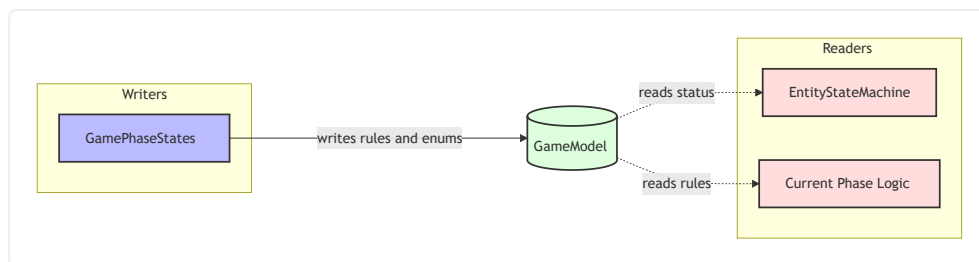
Hierarchisches MVC

Innerhalb des **GameState** gibt es zwei Sub-Systeme, die ebenfalls dem MVC-Prinzip folgen: * **GamePhaseStates**: Repräsentieren die aktuelle

Spielphase (z.B. Green Light, Red Light). Jede Phase hat ihre eigene Logik (Controller), Daten (Model) und Anzeige (View). Der `GameController` entscheidet anhand des Spielverlaufs, welche Phase aktuell aktiv ist und ruft deren `update()` auf. * **Entities**: Jeder Spieler oder NPC wird durch eine `EntityStateMachine` repräsentiert. Diese Maschinen sind in einer Liste im `GameModel` gespeichert. Der `GameController` iteriert über diese Liste und führt für jede Entity ein Update durch.

Interaktion und Datenfluss (Blackboard Pattern)

Dieses Diagramm verdeutlicht die zentrale Rolle des `GameModel` als "Blackboard" bzw. gemeinsamer Datenspeicher.



Funktionsweise des Blackboard Patterns

Das Projekt verwendet das **Blackboard Pattern**, um die Kommunikation zwischen den verschiedenen Steuerungseinheiten (Controllern) und den Akteuren (Entities/Phasen) zu entkoppeln: * **Writers**: Die `GamePhaseStates` schreiben phasen-spezifische Informationen in das `GameModel` (z.B. aktuelle Zeit, Phasen-Enum, Regeln wie "Darf ich mich bewegen?"). * Der **GameController** delegiert das Update an die Phasen und steuert die übergeordnete Entity-Liste. * Die **EntityStateMachines** lesen diesen Status autonom bei jedem Update aus dem Model. * Dies ermöglicht eine saubere Trennung zwischen Welt-Logik und individueller Actor-Logik (Akteure müssen den globalen Controller nicht kennen).

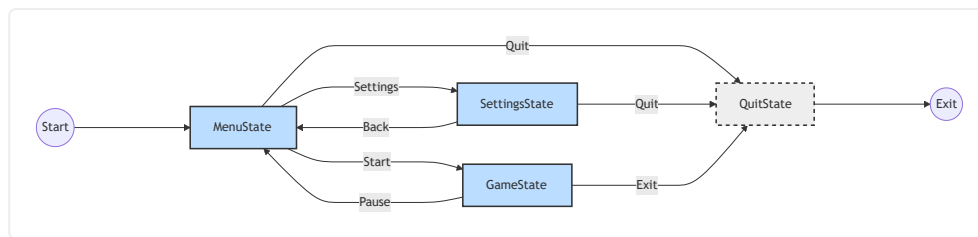
State Machine (FSM)

Der folgende Graph zeigt die Zustandsübergänge der Anwendung (Finite State Machine).

State Machine Diagram (High-Level)

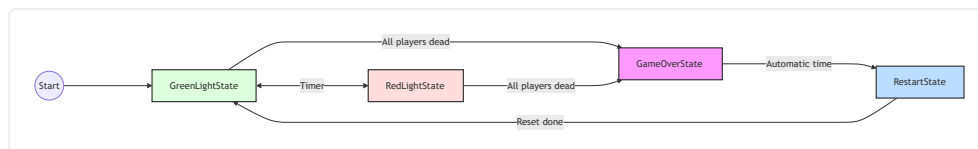
Dieser Graph zeigt die Übergänge zwischen den Haupt-Zuständen der Anwendung. Die `GameApp` sorgt dafür, dass immer genau ein Zustand aktiv

ist.



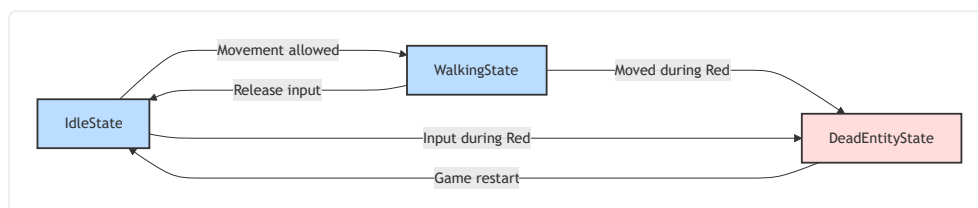
Game Phases (Sub-FSM in GameState)

Innerhalb des `GameState` existiert ein eigenständiger Automat für den Spielablauf. Bemerkenswert ist, dass der Übergang von `GameOver` zu `Restart` ohne Nutzereingabe nach einem Timer erfolgt, um einen flüssigen Spielablauf für BCI-Experimente zu gewährleisten.



Entity State Machine (Player Behavior)

Dieser Automat beschreibt das Verhalten einer einzelnen Spielfigur. Die "Todes-Logik" ist hierbei eng mit dem `GameModel` verknüpft: Wenn das Model signalisiert, dass der aktuelle Phasen-Regelsatz Bewegung verbietet, führt jede Input-Aktion zum sofortigen Wechsel in den `Dead`-Status.



Analyse der Implementierung

Hierarchischer Zustandsautomat (HSM)

Die Anwendung implementiert das Konzept eines **Hierarchischen Zustandsautomaten (Hierarchical State Machine)** durch Komposition und Delegation:

1. **Level 1 (App):** `GameApp` verwaltet die High-Level-Zustände (`Menu` , `Game` , `Settings`).

2. **Level 2 (Game Phases):** Innerhalb von `GameState` existiert ein Sub-Automat für die Spielphasen (`RedLight` , `GreenLight` , `GameOver`). Dieser wird explizit vom `GameController` gesteuert, der die zeitliche Abfolge und Sieg/Niederlage-Bedingungen prüft.
3. **Level 2 (Entities):** Jede Entity besitzt einen eigenen inneren Zustand (`Idle` , `Walking` , `Dead`), der durch die `EntityStateMachine` verwaltet wird.

Kommunikation via Blackboard

Das `GameModel` fungiert als **Shared State Blackboard**: * Die aktiven **GamePhaseStates** schreiben globale Spielparameter (z.B. `is_movement_kills_player`) in das Model. * Der **GameController** delegiert das Update an die Phasen und steuert die übergeordnete Entity-Liste. * Die **EntityStateMachines** lesen diesen Status autonom bei jedem Update aus dem Model. * Dies ermöglicht eine saubere Trennung zwischen Welt-Logik und individueller Actor-Logik (Akteure müssen den globalen Controller nicht kennen).

Dateistruktur (Auszug)

- `Main.py` : Einstiegspunkt.
- `RedLightGreenLight/` : Hauptpaket.
 - `GameApp.py` : Hauptschleife und Manager-Initialisierung.
 - `States/` : Enthält alle State-Implementierungen.
 - `State.py` : Basisklasse.
 - `StateFactory.py` : Erzeugung der States.
 - `Menu/` : MVC für das Menü.
 - `Game/` : MVC für das Spiel.
 - `SettingsSubMenu/` : MVC für Einstellungen.